YourNameMart — Native C++ Marketplace Backend Specification

Drogon · libpqxx/SQLite · PostgreSQL — Native C++20 Equivalent of the Servlets/JDBC Capstone

Checkpoint window: Jul 27 – Oct 10, 2026 · **Builder:** solo

> Replace `YourNameMart` (and `yournamemart` in namespaces, file paths, and DB names below) with a name of your own — e.g. if your name is Aditya, the project becomes `AdityaMart`, namespace `aditya::adityamart`. Apply consistently across the codebase, README, and deployed project title.

---

1. Project Definition & Feature Requirements

Project type: Multi-seller e-commerce marketplace web application ("YourNameMart"), served by a native C++ HTTP application.

Description: Sellers list products. Buyers browse, search, add to cart, and purchase. An admin manages users, orders, and listings.

Scope constraints:

- No real-time infrastructure (WebSockets, live location tracking, external mapping APIs).
- No third-party payment gateway integration. Checkout implements a mock payment confirmation step.

Mandatory feature requirements:

ID	Requirement
F1	User registration and login with two roles: Buyer, Seller. Admin role assigned via a seed row inserted by migration; no separate admin signup flow.
F2	Seller: create, edit, delete product listings (name, description, price, stock quantity, category, image URL).
F3	Buyer: browse and search/filter products by category and keyword.
F4	Cart: add, update, remove items; display running total.
F5	Checkout: place an order from cart contents via mock payment confirmation.
F6	Order history: buyer view of past orders and status; seller view of incoming orders for their products.
F7	Admin: view all users and orders; moderate/remove listings.
F8	Product reviews and star ratings on completed orders.

Optional feature requirements (implement only after F1–F8 are complete and tested):

ID	Requirement
O1	Wishlist / save-for-later.
O2	Order status workflow: Pending → Confirmed → Shipped → Delivered.
O3	Seller sales dashboard (order counts, revenue totals).
O4	AI chatbot — mandatory deliverable for the Final Review (Section 10), sequenced as optional only relative to F1–F8.

Action item — due Jul 27: Initialize the GitHub repository with a working `CMakeLists.txt` and `vcpkg.json` manifest.

---

2. System Architecture Specification

```
Browser (HTML/CSS/vanilla JS + fetch AJAX)
        |  HTTP request
        v
Filter layer      -> AuthFilter (session check), LoggingFilter (request-id), CORSFilter (if needed)
        v                (drogon::HttpFilter subclasses)
Router            -> Drogon's built-in router dispatches to HttpController subclasses
        v
Controller layer  -> ProductController, OrderController ... (thin: no SQL, no business logic)
        v
Service layer     -> business logic, validation (pure C++ classes — no DB calls here)
        v
Repository layer  -> ProductRepository, OrderRepository ... — ALL SQL lives here, parameterized only
        v
Connection Pool   -> owned by a drogon::Plugin, initialized once at process startup
        v
PostgreSQL (production, server mode) / SQLite (embedded, local dev & automated tests)
```

Module/directory structure:

```
yournamemart/
 ├── CMakeLists.txt
 ├── vcpkg.json
 ├── src/
 │    ├── controller/   (HttpController subclasses — thin, no SQL, no
business logic)
 │    ├── service/      (business rules, orchestration — pure C++, no DB
calls)
 │    ├── repository/   (interfaces + libpqxx/SQLite implementations — ALL
SQL lives here)
 │    ├── model/         (POD structs / entities)
 │    ├── dto/           (request/response shapes for JSON endpoints)
 │    ├── filter/        (drogon::HttpFilter subclasses — auth, logging,
encoding)
 │    ├── plugin/        (drogon::Plugin subclasses — DB pool
init/teardown, config load)
 │    ├── util/          (PasswordUtil, ValidationUtil, JsonUtil)
 │    └── exception/     (exception hierarchy, global error translator)
 ├── db/
 │    ├── schema.sql
 │    ├── seed.sql
 │    └── migrations/    (V1__init_schema.sql, V2__..., applied by the
migration runner)
 └── test/               (GoogleTest suites, mirrors src/ structure)
```

Mandatory engineering rules:

1. All SQL uses parameterized queries (`pqxx::exec_params` / prepared statements, or Drogon ORM's built-in binding). No string-concatenated queries, under any circumstance.
2. Passwords hashed with Argon2id via libsodium (`crypto_pwhash_str`). No plaintext storage. No MD5/SHA1-only hashing.

3. Session management via Drogon's built-in `Session` (cookie-backed, server-side store). On successful login, discard the pre-auth session and issue a fresh one. Set an explicit idle timeout.
4. Escape all user-supplied output before rendering. If using Drogon CSP templates, rely on their auto-escaping context; never hand-build HTML by concatenating user input.
5. Connection pool lifecycle owned by a single `drogon::Plugin`, configured once at startup. No ad-hoc connection construction anywhere else in the codebase.
6. RAII for every `Connection`, transaction, and prepared statement — acquired via the pool and released automatically on scope exit, never a manually paired open/close. No raw owning pointers in repository or service code: use `std::unique_ptr`/`std::shared_ptr` exclusively.
7. AddressSanitizer + UndefinedBehaviorSanitizer clean on every CI build. ThreadSanitizer runs at least once per sprint.

---

3. Technology Stack Specification

Component	Specification
Language/standard	C++20, GCC 12+ or Clang 15+
Build tool	CMake ⩾ 3.25
Package manager	vcpkg (manifest mode, `vcpkg.json`)
Web framework	Drogon
Database	PostgreSQL 15+ (production) / SQLite 3 (embedded, local dev & tests)
DB access	libpqxx (Postgres) and the SQLite C API, both via a thin RAII wrapper — or Drogon's built-in `drogon::orm::DbClient`, which speaks both backends through one API
Connection pooling	Drogon's native async `DbClient` pool, initialized inside a `Plugin`
View layer	Drogon CSP (C++ Server Pages — compiled to C++, auto-escaping) for any server-rendered pages; vanilla JS + `fetch()` for AJAX-driven pages
JSON	nlohmann::json for DTO (de)serialization; Drogon's native `Json::Value` (JsonCpp) at the HTTP boundary
Password hashing	libsodium, `crypto_pwhash` (Argon2id)
Testing	GoogleTest + GoogleMock
Logging	spdlog
Static analysis	clang-tidy, cppcheck, clang-format

Component	Specification
Sanitizers	AddressSanitizer + UndefinedBehaviorSanitizer mandatory every CI run; ThreadSanitizer weekly
CI	GitHub Actions — `cmake --build . && ctest` on every push
Outbound HTTP (chatbot)	cpp-httplib or libcurl

---

4. Database Design Specification

Minimum schema:

```
users(id PK, name, email UNIQUE, password_hash, role CHECK(role IN
('BUYER','SELLER','ADMIN')), created_at)
products(id PK, seller_id FK->users, name, description, price_cents BIGINT,
stock_qty, category, created_at)
orders(id PK, buyer_id FK->users, status CHECK(status IN
('PENDING','CONFIRMED','SHIPPED','DELIVERED','CANCELLED')),
total_amount_cents BIGINT, created_at)
order_items(id PK, order_id FK->orders, product_id FK->products, quantity,
unit_price_cents BIGINT)
cart_items(id PK, user_id FK->users, product_id FK->products, quantity)
reviews(id PK, product_id FK->products, user_id FK->users, rating, comment,
created_at)
```

Mandatory design rules:

1. Every foreign key indexed.
2. `email` column unique-constrained.

3. Monetary values stored as `BIGINT` integer minor units (cents). `FLOAT`/`DOUBLE`/`REAL` prohibited for currency fields. Wrap this in a single `Money` value type in `model/` so no arithmetic on raw integers leaks into business logic.
4. `created_at` timestamp present on every table.
5. `schema.sql`, `seed.sql`, and every numbered migration checked into the repository.

---

5. Execution Timeline & Milestones

Window	Dates	Required Output	Checkpoint
Kickoff	Jul 24 – Jul 27	Problem statement locked, repo initialized, CMake/vcpkg/Drogon "hello world" running, DB schema v1	Problem statement due Jul 27
Week 1	Jul 27 – Aug 2	Authentication (register/login/session via Drogon Session), repository layer skeleton, `AuthFilter`, local deploy working	—
Week 2	Aug 3 – Aug 9	Core flow: browse → add to cart → place order	—
—	Aug 10	Working MVP demo	MVP Review
Week 3	Aug 10 – Aug 16	Review feedback addressed; seller dashboard (listing management) started	—
Week 4	Aug 17 – Aug 23	Seller dashboard complete; admin panel started	—
Week 5	Aug 24 – Aug 30	Search/filter, order status workflow	—
Week 6	Aug 31 – Sep 6	Reviews/ratings, edge case handling, input validation	—
Week 7	Sep 7 – Sep 13	Security requirements (Section 8) implemented; ASan/UBSan clean; unit and repository test coverage completed	—

Window	Dates	Required Output	Checkpoint
Week 8	Sep 14 – Sep 20	Deployment finalized (binary + systemd + Nginx TLS); live URL smoke-tested	Deploy link due Sep 21
—	Sep 21	Full build demo on live cloud link	2nd Review
Week 9	Sep 21 – Sep 27	AI chatbot integration — `ChatController` + `ChatService` + widget	—
Week 10	Sep 28 – Oct 4	Chatbot refinement, UI polish, README/architecture documentation	—
Week 11	Oct 5 – Oct 10	Final regression pass, report, slide deck, rehearsed demo	Final Review Oct 10

6. Testing Strategy

Type	Scope	Tool
Unit	Service-layer business logic, isolated via GoogleMock repository interface fakes	GoogleTest + GoogleMock
Repository	Real SQL executed against embedded SQLite `:memory:` or an ephemeral PostgreSQL instance in CI	GoogleTest, run on every push
API/Integration	Full HTTP round-trip against a running Drogon instance	GoogleTest + Drogon's `HttpClient`, or a scripted smoke test against `/api/v1/...`
Load	Concurrent user handling	Apache JMeter, or `wrk`/`ab` — minimum 10 concurrent users, 60 seconds
Memory & concurrency safety	UB, leaks, data races	ASan + UBSan on every CI build; ThreadSanitizer weekly; optional libFuzzer harness on input-parsing code

7. Security Checklist

Complete before Sep 21:

- [ ] Every query parameterized. Verify: `grep -rn "exec(" src/`, confirm each hit uses `exec_params`/a prepared statement, never raw interpolated SQL.
- [ ] Passwords Argon2id-hashed via libsodium, never logged.

- [ ] All protected controllers enforce session checks via `AuthFilter`.
- [ ] User-supplied output escaped before rendering.
- [ ] File upload (if implemented) validates MIME type and size server-side; never trusts a client-supplied filename or extension.
- [ ] Error responses never leak stack traces, exception messages, or internal paths — a global exception handler maps every error to the fixed JSON envelope (Section 11).
- [ ] Database credentials excluded from version control (`config.json`/`.env` in `.gitignore`; `.env.example` committed).
- [ ] Zero AddressSanitizer / UndefinedBehaviorSanitizer findings on the CI build.

---

8. Deployment Specification

Deployment requirements (mandatory, platform-agnostic):

1. Publicly accessible URL, reachable for the duration of the checkpoint window (Jul 27 – Oct 10).
2. The application is a standalone compiled binary.
3. Database (PostgreSQL, or SQLite for a lighter deployment) reachable by the deployed application, with data persisting across restarts.
4. Availability guaranteed during scheduled review sessions (Aug 10, Sep 21, Oct 10).

Reference setup (self-hosting on a Linux VM):

1. Provision the VM — **Week 1**.
2. Install the toolchain: `apt install build-essential cmake ninja-build`, plus vcpkg for dependencies.
3. `cmake --build . --config Release` produces a single `yournamemart` executable.
4. Run it as a `systemd` service (`ExecStart=/opt/yournamemart/bin/yournamemart`).
5. Open port 8080 directly, or put Nginx in front as a reverse proxy on 80/443 terminating TLS via Let's Encrypt/Certbot.

6. Install PostgreSQL as its own systemd service; apply `schema.sql` and run the migration runner (Section 12) once at first deploy.
7. Docker alternative: multi-stage build — a build-stage image with the full toolchain compiles the binary; the runtime image is a minimal base (e.g. `debian:bookworm-slim`) containing only the binary and its runtime shared libraries.
8. Configure uptime monitoring (e.g. UptimeRobot, 5-minute interval). Back up the PostgreSQL data directory (or the SQLite file) after each review.

Backup requirement: Record a 2–3 minute screen-capture demonstration prior to each review, for use if the live deployment is inaccessible during evaluation.

---

9. AI Feature Specification (Phase 3)

Required architecture:

```
Chat widget (JS, floating button + panel)
        |  fetch('/api/chat', POST {message})
        v
ChatController  (validates input, enforces per-session rate limit)
        v
ChatService  -> calls an LLM API server-side via cpp-httplib/libcurl
        v
Returns JSON { reply: "..." } -> widget renders it
```

Requirements:

1. API key read server-side only from an environment variable at process startup. Must not appear in client-side code or in version control.
2. Chatbot scope restricted to product/listing domain queries via a fixed server-side prompt template.
3. Outbound HTTP call wrapped in explicit timeout + error handling. On failure or timeout, return a static degraded response instead of an error page.

4. Per-session rate limit (10 messages/minute) tracked in an in-memory map guarded by a mutex, keyed on the Drogon session ID.
5. Repeated identical questions within a session cached in-memory (`std::unordered_map<std::string, std::string>` per session).
6. Minimum functional scope for Oct 10: correct responses to 5–10 FAQ-style domain questions, with functioning UI integration.

---

10. Coding Standards

Rule	Standard
Naming	Google C++ Style Guide: types `PascalCase`, functions `PascalCase`, member variables `snake_case_` (trailing underscore), constants `kCamelCase`, namespaces `lower_snake_case`
Documentation	Doxygen comments required on every public class and method in `service`/`repository` layers
Static analysis	clang-tidy + cppcheck configured as CMake/CI targets; clang-format enforced pre-commit
Resource safety	RAII everywhere. No raw `new`/`delete` for owned resources. No manual reference counting where `std::shared_ptr` suffices
SOLID	Controllers restricted to HTTP orchestration. Business rules confined to `service`. Services depend on repository interfaces (abstract base classes with pure virtual methods), never on concrete repository classes — dependency inversion via constructor-injected `shared_ptr<Interface>`
Design patterns required	Repository (data access abstraction), Front Controller (Drogon's router + a common base controller), Singleton (connection pool, owned by a `Plugin` and exposed via a thread-safe accessor), Factory (repository/provider instantiation returning `std::unique_ptr<Interface>`), Strategy (swappable notification/payment-mock channels behind an interface, selected by config), Builder (fluent DTO construction), Dependency Injection (manual constructor injection). Document pattern usage in the final report

11. API Contract Standards

1. All JSON endpoints versioned under `/api/v1/...`.

2. Response envelope, fixed format:

```json
{ "success": true,  "data": { ... }, "error": null }
{ "success": false, "data": null,    "error": { "code": "VALIDATION_ERROR", "
```

3. HTTP status codes: 200/201 success, 400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 500 server error. Never return HTTP 200 for an error response.
4. DTOs are separate structs/classes from entities. `UserResponseDto` must not contain `password_hash`.
5. Validation occurs at the top of the service method, before any repository call. Return HTTP 400 with a field-level error on failure.

---

12. Database Migration Requirements

1. Every schema change is a new, numbered, checked-in file: `db/migrations/V1__init_schema.sql`, `V2__add_reviews_table.sql`, `V3__add_index_orders_status.sql`. Never hand-edit a live table.
2. A hand-written migration runner — a `MigrationService` class invoked from a startup `Plugin` — reads `db/migrations/*.sql` in numeric order, checks a `schema_migrations` tracking table for what's already applied, and applies anything new inside a transaction.
3. Seed/demo data maintained in `db/seed.sql`, kept separate from schema migrations.

---

13. Change Management Workflow

1. **Intake:** Every feature or bug originates as a GitHub Issue using a template:

- Feature/Change Request: description, motivation, acceptance criteria, affected layers.
- Bug report: steps to reproduce, expected vs. actual, environment.

2. **Impact analysis:** 3–4 lines on the issue prior to implementation — DB migration required? API response shape changed? Additive or shared-code change?
3. **Branch and PR:** `feature/<n>` branch. PR description states what changed, why, and how it was tested. Checklist: tests added, docs updated, migration included if schema changed.
4. **Versioning:** Semver tags — `v0.1.0` (MVP) → `v0.2.0` … `v1.0.0` (Sep 21 build) → `v1.1.0` (AI chatbot).
5. **CHANGELOG.md:** One line per release, updated as part of the PR that ships it.

---

14. Sprint Execution Cycle

1. Timeline (Section 5) executed as five ~2-week sprints.
2. Each sprint: sprint planning (select 3–5 issues from backlog) → mid-sprint check-in at day 7 → sprint review against Definition of Done (Section 17) → retro entry in `RETRO.md` (one line: what worked, what didn't, one change for the next sprint).
3. Work tracked on a GitHub Projects Kanban board: `Backlog → In Progress → In Review → Done`.

---

15. AI Integration Architecture Requirements

1. AI provider accessed through an interface, not a hardcoded implementation:

```cpp
```

```cpp
class ChatProvider {
public:
    virtual ~ChatProvider() = default;
    virtual std::string GetReply(const std::string& user_message,
                                 const std::string& context) = 0;
};
// GeminiChatProvider : public ChatProvider { ... real API call via libcurl/c
// MockChatProvider    : public ChatProvider { ... canned FAQ answers, no net
```

2. Implementation selected via a config flag (`ai.chatbot.provider = gemini|mock`), wired up through the Factory pattern from Section 10.
3. Guardrails required: per-session rate limit (10 messages/minute), input length cap, timeout on the outbound API call, fixed server-side prompt template.
4. Repeated identical questions cached in-memory per session.

---

16. Observability & Environment Requirements

1. `GET /api/v1/health` returns `{ "status": "UP", "db": "UP" }`, verifying database connectivity by executing a trivial query (`SELECT 1`).
2. Structured logging via spdlog: a request ID is generated per incoming request in `LoggingFilter` and passed explicitly through the controller → service → repository call chain, included in every log statement for that request.
3. `.env.example` committed; the real `.env`/`config.json` excluded via `.gitignore`, documenting every required config value.
4. `CONTRIBUTING.md`: exact steps from `git clone` through `vcpkg install` → `cmake --preset ... && cmake --build ...` → running the binary locally.

---

17. Definition of Done

Applies to every feature prior to merge:

- [ ] Compiles with no clang-tidy/cppcheck warnings at the configured severity
- [ ] Zero AddressSanitizer/UndefinedBehaviorSanitizer findings
- [ ] Unit/repository tests written and passing
- [ ] Code self-reviewed prior to merge
- [ ] Migration script included if schema changed
- [ ] Verified against the deployed URL, not only localhost
- [ ] README/API documentation updated if behavior changed
- [ ] Merged to `main` only when CI is green

This specification is binding for the checkpoint window Jul 27 – Oct 10, 2026.